Q1 Ideation: PM workflow pain points and agent solutions

Prepared by: AI PM/TPM, Product and Innovation team, NeuronForge Technologies Date: 28 October 2026 Status: Draft for review

1. Summary

NeuronForge's PMs and TPMs turn meeting discussions into PRDs, epics and user stories by hand. The work is slow, every PM formats it differently, and requirements that were said in a meeting are often lost before they reach engineering. This document lists five pain points in that workflow. For each one it gives the manual step, the agent that supports it, the inputs and outputs, and the risks.

One design rule runs through all of it: grounding. Every item an agent produces must trace to something a person actually said, and anything an agent adds on its own is labeled as a draft for the PM to accept or reject.

#	Pain point	Manual step today	Agent solution
1	Gathering scope from meetings is slow	Re-reading transcripts and notes, copying requirements into a document	Requirement Extractor
2	Turning discussions into a PRD is manual and iterative	Writing each PRD from scratch, in the PM's own style	PRD Generator
3	PRD formats differ, which makes estimation hard	Engineering reformats or interprets each PRD before estimating	One fixed template with stable ids
4	Requirements get lost and gaps surface late	Notes sit in documents; gaps are found during the build	Gap Analyzer
5	Breaking a PRD into epics and stories is manual, and stories lose their source	Writing stories and criteria by hand	Story Breakdown agent

2. Who is affected, and how the work flows today

Primary users are the PMs and TPMs in the Product and Innovation team who run requirement meetings. Engineering leads are secondary users, because they read the PRDs and estimate from them. Designers and the business stakeholders who are interviewed are affected too.

Today the flow is:

1. A meeting or interview happens, and a transcript or notes are produced.
2. The PM re-reads the material, marks what sounds like a requirement, and follows up on unclear points.

3. The PM writes a PRD in their own format.
4. The PM breaks the PRD into epics, features and user stories.
5. Engineering reads the PRD, asks for clarification, and estimates.
6. Changes loop back to step 2.

3. Pain points

Pain point 1: gathering scope and requirements from meetings takes too long

- Manual step: The PM re-reads the transcript or notes, highlights what sounds like a requirement, copies it into a working document, and checks names, numbers and deadlines by hand.
- Why it hurts: The work is repetitive and error-prone. A requirement mentioned once in a long meeting is easy to miss, and a number copied wrongly can go unnoticed.
- Agent solution: The Requirement Extractor reads the text and returns only what was stated. Each requirement comes with its exact quote and speaker. It also returns acceptance criteria where they were stated, stakeholders, deadlines, dependencies, assumptions that were explicitly framed as assumptions, ambiguous items, contradictions and risks. It decides whether the input is sufficient, and if nothing concrete was said, it says so and the pipeline stops.
- Inputs: Meeting transcripts, stakeholder notes or a product brief, pasted as text or uploaded as files (.txt, .md, .docx, .pdf).
- Outputs: A structured extraction with requirement ids (REQ-001 and so on), each tied to a verbatim quote, plus a sufficiency flag.
- Risks and guardrails:
    - The agent invents a requirement that was never said. Guardrail: every requirement needs a verbatim quote, and numbers are copied exactly.
    - The agent misses a requirement. Guardrail: the PM reviews the extraction before the PRD is generated.
    - Vague input produces confident-looking output. Guardrail: a sufficiency gate blocks PRD generation when no concrete requirement exists.

Pain point 2: turning discussions into a PRD is manual and iterative

- Manual step: The PM starts from a blank page, restates the requirements in prose, and fills sections such as goals, personas and timeline, often guessing at what the meeting did not say.
- Why it hurts: Drafts take a long time, and guesses read like facts once they are written down.
- Agent solution: The PRD Generator fills a fixed 10-section template using only the approved extraction. Where the source is silent, the field says Not specified in source. Priority is set only when a priority was stated, and the date and version are supplied by the system.
- Inputs: The approved extraction, plus the PM's answers to any gap questions.
- Outputs: A PRD in Markdown, with functional and non-functional requirement tables that cite their source requirement ids.
- Risks and guardrails:
    - Filler content that looks authoritative. Guardrail: the Not specified in source rule and a ban on adding goals, metrics or personas that were not stated.

    - A wrong priority. Guardrail: priority comes only from stated signals; otherwise it is marked Unprioritized.
    - The PM trusting a polished document. Guardrail: human sign-off before the next stage.

Pain point 3: PRD formats differ, which makes estimation hard

- Manual step: Engineering receives PRDs in different layouts and has to hunt for requirements, targets and dependencies, or ask the PM to restate them before estimating.
- Why it hurts: Estimation is slower and results are hard to compare across products.
- Agent solution: Every PRD uses the same template, the same section order and the same ids (FR-001 for functional and NFR-001 for non-functional requirements). The PM edits content, not structure.
- Inputs: The approved extraction.
- Outputs: A consistent PRD. Format compliance (the share of PRDs that match the template) becomes a measurable number.
- Risks and guardrails:
    - The template is too rigid for some products. Guardrail: a versioned template that the PM can edit, with empty sections shown explicitly instead of removed.

Pain point 4: requirements get lost, and gaps surface late

- Manual step: Notes stay in a document nobody revisits. The PM notices missing information or conflicting positions only when a developer asks, sometimes weeks later, and writes follow-up questions ad hoc.
- Why it hurts: Lost requirements cause rework, and late questions slow the whole project.
- Agent solution: The Gap Analyzer reads the extraction and writes one specific question per gap: missing information, ambiguity, contradiction or dependency risk. Each has a severity, and a suggested owner only when the owner was named in the source. Contradictions and dependencies with an unknown ETA rank first. The extraction itself also keeps every stated requirement with its quote, so a meeting's outcome stays searchable instead of buried.
- Inputs: The extraction.
- Outputs: An ordered list of clarification questions for stakeholders.
- Risks and guardrails:
    - Noisy or generic questions. Guardrail: no gaps for information already stated, and one question per gap.
    - Missed conflicts. Guardrail: coverage rules and PM review. This was a real finding in testing: an explicit disagreement between two people was caught, but an implicit trade-off inside one statement was missed.

Pain point 5: breaking a PRD into epics and stories is manual, and stories lose their source

- Manual step: The PM splits the PRD into epics, features and stories, writes the As a / I want / So that text, adds a priority and acceptance criteria, and tries to keep each story tied to the PRD.
- Why it hurts: It is time-consuming, and the link from a story back to the original requirement is easily lost.
- Agent solution: The Story Breakdown agent writes epics, features and stories from the approved PRD: one story per capability and per persona, persona names taken from the PRD, a priority

suggestion with its rationale, acceptance criteria copied verbatim when the PRD states them, and non-functional targets attached to the relevant story. Anything the agent proposes on its own, such as a benefit or a business rule, is tagged [DRAFT] so the PM can accept or reject it.

- Inputs: The approved PRD.
- Outputs: Epics, features and stories in Markdown, with source ids and INVEST flags.
- Risks and guardrails:
    - Invented specifics. This was observed in testing: the agent proposed default values and exact message texts that no one had stated. They were tagged as drafts, but they still need review. Guardrail: tags, PM review, and a planned step where the PM answers decision questions before stories are written.
    - Merged personas. Guardrail: a rule to write separate stories per persona.

The outcome: delays and inconsistent tracking

The delays and inconsistent tracking named in the problem statement are the result of the five pain points above, not a separate manual step. They are addressed by traceable ids (REQ, FR, NFR and story ids) and by one consistent set of artifacts. They are measured by the time from meeting to an approved PRD.

4. Risks that apply to every agent

Risk	Why it matters	Mitigation
Hallucinated requirements	Engineering builds the wrong thing	Verbatim-quote rule, sufficiency gate, evaluation on a fixed baseline set, human sign-off
Missed requirements or conflicts	Silent scope gaps	Gap Analyzer, PM review of the extraction, regression tests
Over-trust in a polished document	The PM approves without reading	[DRAFT] tags, Not specified in source markers, a review status on every story, approval between stages
Confidential content sent to a third-party model	Meeting notes can hold customer or strategy information	Enterprise API terms, redaction before upload, access control, a data-handling decision by Security
Model or prompt changes break behavior	Quality drifts without anyone noticing	Fixed baseline test set, tracing, re-running tests before each prompt change
Poor source material	Bad transcription or mixed topics lead to bad output	Sufficiency flag, PM check, a request for better input

5. What the prototype showed

These are results from the baseline tests of the working prototype and from human annotation of nine traces. Faithfulness averaged 4.9 out of 5 and completeness 4.7, but usefulness only 3.3. One trace (T6) contained an unsupported detail. The agents rarely invent; they sometimes fail to flag or capture something, such as a conflict (T3), a named person (T5) or which API is meant (T7).

- Vague or empty input was flagged as insufficient and was not turned into requirements.
- Exact figures were preserved: 10,000 concurrent users, 200 ms at p95, Salesforce REST API v52.
- An explicit disagreement (microservices versus a single-page app) was found and raised as a high-severity question. An implicit trade-off (frequent refresh versus minimizing API calls) was missed.
- The Stories stage is the weakest. It added invented defaults and message texts, tagged as drafts, and ignored some formatting rules.
- A full run (Analyze, PRD, Stories) takes about 36 seconds and costs about $0.002. A complete document lifecycle with two PRD revisions costs about $0.003.

6. Prioritization

Rank	Pain point	Impact	Feasibility	State in the prototype
1	Gathering scope from meetings	High	High	Working
2	PRD generation and consistent format (pain points 2 and 3)	High	High	Working
3	Lost requirements and late gaps	High	Medium	Partly working; implicit conflicts are missed
4	Epics and stories	Medium to high	Medium	Weakest stage; needs a questions-first step

Not in version 1: market research, effort estimation, and direct integration with tools such as Jira or Confluence. They are roadmap items.

7. Assumptions to validate with PMs

- PMs spend a large share of requirement-gathering time re-reading and reformatting. To validate: measure the time per PRD in a baseline survey.
- Engineering finds inconsistent PRD formats a real cause of estimation delay. To validate: ask engineering leads.
- PMs will review drafts and not approve them blindly. To validate: observe the pilot.
- Source material is available as text or Word documents, not only as audio.